

Next-Gen Read Alignment and Contig Query Identification

By: Christian Anderson

26 March 2026

[GitHub](#)

Contents

1. Timeline	2
2. Findings	2
2.1. Read Alignment	2
2.2. Main Algorithm	4
3. Conclusions, Thoughts, and Next Steps	6
4. Updated timeline	6
5. Algorithm Code	8

1. Timeline

My goals for the second module of this project were as follows:

- *Brainstorm Algorithm* ✓
 - *Research De Bruijn Graphs* ✓
- *Develop program* ✓
 - *Load Data* ✓
 - *Parse fastas into workable python objects* ✓
 - *Fragment reads into kmers* ✓
 - *Save kmer read ids* ✓
 - *Filter Kmers* ✓
 - *Make directed graph* ✓
 - *Connect similar kmers* ✓
 - *make adjacency matrix where k-1 overlap* ✓
 - *Make contigs* ✓
 - *Traverse every path from some set of starting nodes* ✓
 - *Construct contig along the way* ✓
 - ***Save contigs containing query sequence***
- *Create a Github repo* ✓

Since my initial brainstorming for this project summarized in my Module-1 talk, I have completed nearly all of my goals (checkmarked, ✓). I still need to identify contigs containing the query sequence due to my code's current limitations (*Section 3*).

2. Findings

Here I go in to depth outlining my approach to find the longest contig containing a query sequence. As of yet, I do not have results but I have learned a lot and have much to improve on.

2.1. Read Alignment

The first steps of loading and parsing the fastas was trivial, but fragmenting the kmers was slightly more complicated as I wanted a way to encode read frequency and provenance (their parent read). A python `Counter()` was perfect for this as it records the kmer string as well as how many times it was observed. Within the counter loop, I fragment each read into k length words, *mers*, using a stride of one character at a time:

```

for  $i \in [1, |\text{string}| - k]$ , do
     $\text{kmer}_i = \text{string}(i, i + k)$ 
    count  $\text{kmer}_i$ 
    record read id
end for

```

For example with the following string a k of 3:

thequickbrownfox $\rightarrow \text{the}_{i=1}, \text{heq}_{i=2}, \text{equ}_{i=3}, \text{qui}_{i=4}, \text{uic}_{i=5}, \dots, \text{fox}_{i=(|\text{string}| - k)}$

To record kmer provenance, I just ran the kmer counter alongside a dictionary for each kmer which saved the read id and its coordinates in that read which I pass on to the final results. I then took this set of strings and their frequencies and kept ones observed more than once.

The next step was to encode relationships of overlap between these kmers. I accomplished this by looping over every i, j combination of kmers, recording their $k-1$ overlap in an adjacency matrix, A .

$$(A)_{ij} = \begin{cases} 1 & \text{if } \text{kmer}_i(2:k) \equiv \text{kmer}_j(1:k-1) \\ 0 & \text{if } \text{kmer}_i(2:k) \not\equiv \text{kmer}_j(1:k-1) \end{cases} \text{ for } i \neq j$$

The adjacency matrix, A , is the matrix representation of a graph that encodes how each kmer overlaps or doesn't overlap with every other kmer: if for a element with a value greater than zero, that means there is an edge between the column and row kmer. And because I compared every i th to every j th string, each pair of kmers is compared two times, one time A to B and again B to A . This makes the graph directional which is important because DNA is directional, so edges lying below the diagonal ($(A)_{i < j} < 1$) means the 2-length contig reads $\text{kmer}_i \rightarrow \text{kmer}_j$ and edges in the upper diagonal read $\text{kmer}_j \rightarrow \text{kmer}_i$.

Contiguous Sequence Creation

To reiterate this project's primary task, I need to find the DNA sequence containing a query sequence from fragments of DNA. At this point I knew where each kmer came from and how each one overlapped with every other, but translating that overlap information into an actual assembled sequence remained the core challenge. To find the largest contiguous sequence (contig) containing the query sequence requires traversing every possible path through the graph which quickly becomes NP-hard as each step through the graph spawns even more possible paths.

For example, a fully connected graph containing only 10 nodes, P , has $(P - 2)!e = 1.10 * 10^5$ different possible paths from each node to every other node. When I processed the assigned 124,520 reads to a kmer length of 120, I was left with 2,734,978 unique kmers. This number

luckily reduces down to 14,758 when keeping only kmers observed more than three times, but even with a fraction of the total possible kmer count, assuming the graph is fully connected, there would be essentially infinite possible paths from each node to every other (approx. $5.89 * 10^{55111}$). That being said, it was critical that my pipeline reduce the search space and that my algorithm be both memory efficient and fast.

My first idea was to use a recursive algorithm that looks at the next connected nodes for each node, setting each next node as the current, and so on. I set it to loop over nodes with the fewest incoming edges ($\arg \min(\text{colsum}(A))$) and build contiguous sequences from there, keeping track of visited nodes so it didn't get caught in cycles. This approach worked well on test data, but quickly reached recursion limits with the actual assigned data. Recursive algorithms are also mind bending (at least for me), especially when debugging, so I instead used an iterative approach where the algorithm continues searching while possible. This method explores branches in a similar manner (depth first), but it makes the current states more observable, which is preferable for debugging. I'll go through its steps in more depth.

2.2. Main Algorithm

python

```

1 def make_contigs_from_adj(query, adj):
2     def _get_starts(adj):
3         col_sums = adj.sum(0)
4         return np.argwhere(col_sums == col_sums.min()).ravel().astype(np.int32)
5     def _get_nexts(curr):
6         return np.argwhere(adj_np[curr, :] > 0).ravel().astype(np.int32)
7     def _save(contig):
8         """check if contig inds contain teh query string, if so save"""
9         # append last char consecutively
10        string = construct_string_from_debrujin(str_index, contig)
11        if query in string:
12            contig_arr = np.array(contig)
13            contig_res.append((string, contig_arr))
14
15    str_index = adj.columns.values
16    adj_np = (adj > 0).to_numpy().astype(np.int8)
17    starts = _get_starts(adj_np)
18    contig_res = []

```

Code 1: Defining functions and formating initial conditions.

The function accepts a query and adj argument. I made the addition of the query to allow for more dynamic contig creation; because we only care about contigs with the query, the funtion only returns those.

I first define some functions:

- `_get_starts()` finds the starting nodes `starts` which are the indices of the minimum adj colsums.
- `_get_nexts()` finds the next nodes which are the indices of where a current nodes values are greater than 0.

I then converted the adjacency matrix `adj` from a pandas `DataFrame()` to a 8-bit binary numpy matrix to save memory, find the `starts` nodes, and define an empty list to save results.

python

```

1 for start in starts:
2     states = [(start, [start], {start})]
3     while states:
4         current, contig, visited = states.pop()
5         nxts = _get_nexts(current)
6         if len(nxts) == 0:
7             _save(contig)
8             continue
9         for nxt in nxts:
10            if nxt in visited:
11                _save(contig)
12                continue
13            states.append((nxt, contig + [nxt], visited | {nxt}))

```

Code 2: Creating contigs while walking through graph.

Next, I enter the meat of the algorithm where contigs grow as new nodes are visited. Here I utilize a stack, `states`, which grows and shrinks as branches grow, terminate, or are saved. The stack saves the current node in the walk, `start`, the current walk, `[start]`, and the nodes visited along the walk, `{start}`, for all traversed paths from a starting node. The loop begins with `start` in each slot of the state list and then builds that list while `states` is non-empty. For each iteration within the while loop, the most recent (aka, rightmost/topmost) state in the stack is assigned to respective objects and removed from list in the same line once using the `pop()` function. This current node is now being built off of using the `contig` list, so long as there are connected nodes and they are unvisited. If a `nxt` node has been visited or if there are no outgoing edges from the current node, `_save()` will save the contig string and it's indices (kmer ids) to the `contig_res` list only if the query string is in that contig. If there are more `nxt` nodes to visit from the current node, a new state is added to the top of the stack with `nxt` taking the current place, `contig += nxt` taking the place as `contig`, and `visited += nxt` taking the place of the `visited` slot (Code 2, line 13). Here is the critical point where contigs grow as new nodes are visited until there are no more to visit. The stack `states` is constantly growing and shrinking; growing as many `nxt` nodes are visited, and shrinking each time the stack is popped and the contig saved.

This runs for all starts and then returns the `contigs_res` list with the following structure:

```
1 list[ tuple[ string[<contig>], array[<kmer_indices>] ] ]
```

The full algorithm can be found in Section 5.

3. Conclusions, Thoughts, and Next Steps

I've poured 100% effort into the first 90% of this project, therefore last 10% should only cost me 1,000% of my effort. Nevertheless, the most daunting step of designing a graph search algorithm is for the most part completed and the mostly entails playing around with parameters and optimizing the code.

I need to find the sweet spot for the k parameter because as k increases, the number of unique kmers increases as well. To illustrate this, I ran my algorithm twice on the assigned reads. There are 1,008,559 unique 10-mers which reduces down to 810,182 after filtering out uncommon kmers (count < 3) and 2,734,978 unique 120-mers which reduces down to 14,758 after the same filtering. So, as k increases, raw kmer count increases but filtered kmer count decreases. That means I need to gauge the filtering threshold inversely to k and find the region of k and t values that has enough granularity for the query to be findable and computationally feasible.

I am also not utilizing the query to reduce the search space. I could be subsetting the graph for nodes connected to the query's kmers. Starting at the last kmer in the query, I can then branch out from there for every next node to build the kmer from the positive end. I can also branch the other direction to build from the negative end. This approach should not take too much to implement into my existing code and should reduce runtime significantly.

As of right now I calculate an adjacency matrix which is major memory hog. I can try using a dictionary approach to measure each node's preceding and next nodes. Using this should be just as easy and fast as the adjacency matrix to look up the next and preceding kmers without as much overhead.

4. Updated timeline

- Build code (+2 weeks)
 - Optimize code by trying query search cheat
 - Try the dictionary approach to speed up graph search
 - Connect kmers to reads, kmers to contigs, contigs to reads
 - Find the longest contigs with the query sequence
- Find longest contig (+1 weeks)

- Optimize filtering threshold and k parameters
 - Do simple grid search to find right pair
 - Identify simple heuristic for for k and t by trying gridsearch on simulated data
- Finalize GitHub (1 day)
- Complete report (2 days)
- Final presentation (4 days)

5. Algorithm Code

For all codes visit my [GitHub](#).

Code

python

```
1 def construct_string_from_debrujin(kmers:NDArray, inds: NDArray):
2     """Construct continuous string for debrujin graph path, appending every last
    character to string"""
3     string = kmers[inds[0]]
4     for ind in inds[1:]:
5         string+=kmers[ind][-1]
6     return string
7
8 def make_contigs_from_adj(query, adj):
9     def _get_starts(adj):
10         col_sums = adj.sum(0)
11         return np.argwhere(col_sums == col_sums.min()).ravel().astype(np.int32)
12     def _get_nexts(curr):
13         return np.argwhere(adj_np[curr, :] > 0).ravel().astype(np.int32)
14     def _save(contig):
15         """check if contig inds contain teh query string, if so save"""
16         # append last char consecutively
17         string = construct_string_from_debrujin(str_index, contig)
18         if query in string:
19             contig_arr = np.array(contig)
20             contig_res.append((string,contig_arr))
21
22     str_index = adj.columns.values
23     adj_np = (adj > 0).to_numpy().astype(np.int8)
24     starts = _get_starts(adj_np)
25     contig_res = []
26
27     for start in starts:
28
29         states = [(start, [start], {start})]
30
31         while states:
32             # get current states
33             current, contig, visited = states.pop() # take the last element of the
list ( a tuple )
34             nxts = _get_nexts(current)
35
36             if len(nxts) == 0:
37                 _save(contig)
38                 continue
39
40             for nxt in nxts:
41                 if nxt in visited:
42                     _save(contig)
```



```
43         continue
44         # update states pushing forward through tree
45         #++ current:=nxt, contig:= contig+[nxt], visited:= visited + nxt
46         #++ each item of states is a tuple encoding the [current, extending
branch, and visited nodes]
47         states.append((nxt, contig + [nxt], visited | {nxt}))
48
49     return contig_res
50
```

- A well documented and organized GitHub repository.
- A written report containing
 - An update on the proposed timeline
 - Full write ups of completed components. These sections should include details on the solution - and analysis of results.
 - Stubs for partially completed components